CSA1735 – Artificial Intelligence

CO1 Assessment 1 (AT1)

Student: Murali Krishnan S | **Register No:** 192321093

Slot A | Questions: 3, 5, 8, 11, 13

Q3. Exam Timetabling (Constraint Satisfaction Problem)

CSP formulation

- **Variables:** one per exam, X = {M (Maths), P (Physics), C (Chemistry), S (CS), E (English)}.
- **Domains:** time slots D = {1, 2, 3} for every exam.
- **Constraints:** two exams that share at least one student must not be in the same slot. Assumed conflicts: M≠P, M≠S, P≠C, C≠S, S≠E. Further hard constraints can be added, such as room capacity and invigilator availability; soft constraints include minimising back-to-back exams for a student.
- **Goal:** a complete assignment that satisfies all constraints.

Constraint graph: nodes are exams and edges are conflicts (M–P, M–S, P–C, C–S, S–E). This is graph colouring where slots are colours.

Solving by backtracking with MRV, degree heuristic and forward checking

Step	Choice	Reason	Domains after forward checking
1	S = 1	S has the highest degree (3)	M{2,3}, C{2,3}, E{2,3}, P{1,2,3}
2	M = 2	M has the smallest domain (MRV)	P{1,3}, C{2,3}, E{2,3}
3	C = 2	smallest domain; consistent with M, S	P{1,3}, E{2,3}
4	P = 1	consistent with M=2 and C=2	E{2,3}
5	E = 2	consistent with S=1	–

Solution: S = 1, M = 2, C = 2, P = 1, E = 2. Check: M≠P (2,1), M≠S (2,1), P≠C (1,2), C≠S (2,1), S≠E (1,2) → all satisfied. Only 2 slots are needed (chromatic number 2).

Notes: If a domain becomes empty, the algorithm backtracks. Arc consistency (AC-3) can prune domains before and during search. Min-conflicts local search is a good alternative for large timetables.

Q5. 8-Puzzle with Heuristic Search

Formulation

- **States:** location of the 8 tiles and the blank on the 3×3 board (9!/2 = 181,440 reachable states).
- **Actions:** move the blank Up, Down, Left or Right.
- **Transition model:** swap the blank with the adjacent tile.
- **Goal test:** board matches the goal configuration.
- **Path cost:** 1 per move.

Heuristics

- **h1 = number of misplaced tiles** (blank not counted).
- **h2 = sum of Manhattan distances** of tiles from their goal squares.
- Both are admissible; h2 ≥ h1 for every state, so **h2 dominates h1** and expands fewer nodes.

Worked example (assumed)

Start	Goal
2 8 3 / 1 6 4 / 7 _ 5	1 2 3 / 8 _ 4 / 7 6 5

h1(start) = 4 (tiles 2, 8, 1, 6). h2(start) = 1 + 2 + 1 + 1 = **5**.

A* search with f = g + h2

Move	Board (rows separated by /)	g	h2	f
Start	2 8 3 / 1 6 4 / 7 _ 5	0	5	5
Blank up	2 8 3 / 1 _ 4 / 7 6 5	1	4	5
Blank up	2 _ 3 / 1 8 4 / 7 6 5	2	3	5
Blank left	_ 2 3 / 1 8 4 / 7 6 5	3	2	5
Blank down	1 2 3 / _ 8 4 / 7 6 5	4	1	5
Blank right	1 2 3 / 8 _ 4 / 7 6 5	5	0	5

Solution length = 5 moves (optimal). f stays at 5 along the path, as expected with a consistent heuristic. Greedy best-first uses only h (fast but not guaranteed optimal); A* is complete and optimal; IDA* gives the same result using much less memory. Half of the possible start states are unsolvable (parity check on inversions).

Q8. Uninformed Search Comparison (BFS, DFS, IDS)

Let b = branching factor, d = depth of the shallowest goal, m = maximum depth of the tree.

Property	BFS	DFS	IDS
Strategy	Shallowest node first (FIFO queue)	Deepest node first (LIFO stack)	Repeated depth-limited DFS with limit 0, 1, 2, …
Complete?	Yes (finite b)	No in infinite or looping spaces	Yes (finite b)
Optimal?	Yes, if step costs are equal	No	Yes, if step costs are equal
Time	O(b^d)	O(b^m)	O(b^d)
Space	O(b^d)	O(bm)	O(bd)

Key points

- BFS finds the shallowest solution but needs exponential memory.
- DFS uses little memory but can follow deep or infinite branches and return a non-optimal path.
- IDS combines BFS completeness and optimality with DFS memory use. The repeated work is small because most nodes are at the deepest level: total ≈ (d+1) + d·b + … + b^d = O(b^d).
- IDS is the preferred uninformed method when the space is large and the solution depth is unknown.

Q11. Water Jug Problem

Assumed problem: a 4-litre jug and a 3-litre jug with no markings and an unlimited water supply. Measure exactly 2 litres in the 4-litre jug.

Formulation

- **State:** (x, y), where x = water in the 4 L jug (0–4) and y = water in the 3 L jug (0–3).
- **Initial state:** (0, 0). **Goal:** (2, y) for any y.

- **Actions / production rules:**
  a. Fill the 4 L jug: $(x, y) \rightarrow (4, y)$ if $x < 4$
  b. Fill the 3 L jug: $(x, y) \rightarrow (x, 3)$ if $y < 3$
  c. Empty the 4 L jug: $(x, y) \rightarrow (0, y)$ if $x > 0$
  d. Empty the 3 L jug: $(x, y) \rightarrow (x, 0)$ if $y > 0$
  e. Pour 4 L → 3 L until the 3 L jug is full: $(x, y) \rightarrow (x - (3 - y), 3)$ if $x + y \geq 3$, $x > 0$
  f. Pour 3 L → 4 L until the 4 L jug is full: $(x, y) \rightarrow (4, y - (4 - x))$ if $x + y \geq 4$, $y > 0$
  g. Pour all of the 4 L jug into the 3 L jug: $(x, y) \rightarrow (0, x + y)$ if $x + y \leq 3$
  h. Pour all of the 3 L jug into the 4 L jug: $(x, y) \rightarrow (x + y, 0)$ if $x + y \leq 4$
- **Path cost:** 1 per action.

Solution (found by BFS, 6 steps)

Step	State (x, y)	Action
0	(0, 0)	Start
1	(0, 3)	Fill 3 L jug
2	(3, 0)	Pour 3 L into 4 L
3	(3, 3)	Fill 3 L jug
4	(4, 2)	Pour 3 L into 4 L until full
5	(0, 2)	Empty 4 L jug
6	(2, 0)	Pour 3 L jug (2 L) into 4 L jug → **goal**

An equally short alternative: (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3). BFS guarantees the minimum number of steps; DFS may find a longer path or loop unless repeated states are checked.

Q13. 8-Queens Problem (Formulation and Search)

Goal: place 8 queens on an 8×8 board so that no two share a row, column or diagonal.

Formulation (incremental)

- **States:** any arrangement of 0 to 8 queens with none attacking another (placed column by column).
- **Initial state:** empty board.
- **Actions:** add a queen to the leftmost empty column in a square not attacked by any existing queen.
- **Transition model:** the board with the queen added.
- **Goal test:** 8 queens on the board, none attacked.
- **Path cost:** irrelevant; only the final state matters.

State-space size: the naive complete-state formulation has about 1.8 × 10^14 states. The incremental no-attack formulation reduces this to **2,057 states**.

Search

Backtracking (DFS with constraint checking): place a queen in column 1, then column 2 in a safe row, and so on. If a column has no safe row, backtrack and try the next row in the previous column.

```
solve(col):
  if col == 8: return true
  for row in 0..7:
    if safe(row, col):
      place queen at (row, col)
      if solve(col+1): return true
      remove queen
  return false
```

safe(row, col) checks that no earlier queen shares the row or either diagonal.

One solution (row of the queen in columns 1–8): **1, 5, 8, 6, 3, 7, 2, 4**

Facts: 92 distinct solutions, 12 unique up to rotation and reflection. Alternatives: BFS (memory heavy), hill climbing with min-conflicts (fast but can get stuck in local minima, fixed by random restarts or simulated annealing), and genetic algorithms.